You'd continue with this process until you're confident that your class does everything it needs to do to satisfy the acceptance criteria you're working on.

The exact number of unit tests you need to write will vary from case to case. You could repeat this process for each status level, including edge cases, boundary conditions, and so on. This would make troubleshooting easier, but it could introduce some duplication between the acceptance tests and the unit test because the status levels are also described in the acceptance criteria; if the requirements change for the acceptance tests, the unit tests would need updating as well. Alternatively, if the `StatusService` used a database table, and the logic was identical for every status level, you might simply illustrate one status level and let the acceptance test do the work for the others.

Wording your requirements well: low-level BDD vocabulary

As you may have noticed in the examples so far, BDD practitioners often like to express requirements using the word "should." For example,

```
"a new frequent flyer should have Bronze status"
```

or

```
"should be able to upgrade a frequent flyer status"
```

This vocabulary isn't an accident. Although some developers prefer more definitive wording like "must" or "will," and produce fine executable specifications and living documentation doing so, the reasoning behind the BDD "should" are worth considering.

Language influences thought patterns. The words you use influence the way your message is perceived. Consider the following subconscious dialogue that our brains go through when reading a requirement:

> Requirement: A new Frequent Flyer member status must be Bronze.
>
> Brain: Yes Sir, very good Sir, right away Sir!

or

> Requirement: A new Frequent Flyer member status should be Bronze.
>
> Brain: Should it? Are there times when it might not be Bronze? What about special deals from marketing where they start out as Silver? What about if they're transferring points from a partner airline?

Only one word has changed, but the way we receive the information is quite different. The word "should" invites a question: "should it?" "Should" helps us justify why we want a class to provide a particular service, or to perform a particular task. Should also implies that any requirement, at any level, can be questioned, leaving communication channels much more open.

This use of the word "should" is, incidentally, quite different from the way the word is used in more traditional requirements documentation. In old-style formal specifications, and in legal documents, words like "must," "should," and "may" often have very precise meanings. For example, in the IETF recommendations, "must" or "shall"